Alibaba Travel Ticket Reservation System – Phase 2

Alireza Nobakht **Mohammad Taghi Ghaffari** **Mahan Nojavan**

System Overview

This project builds upon the core travel reservation system by integrating advanced analytical functionality and performance optimization features. Users interact with the system to reserve, cancel, and report issues with travel tickets across three transportation modes: bus, train, and plane. Administrative users (support staff) can view complaints, track cancellations, and access dashboards. Phase 2 adds functionality for data-driven insights to support business decisions, as well as efficient data retrieval via indexing and stored procedures.

Note: In this project Data has been created by Fakers

Database Design Summary

Entity-Relationship Design Enhancements

Relationships established between ticket, vehicle, and mode-specific tables like bus, plane, train through polymorphic join logic. Each ticket is connected to a reservation, which is linked to a passenger derived from the general person table. Report table tracks user-submitted issues against tickets, while support staff respond and resolve them.

Normalization to 3NF

All attributes are fully dependent on the primary key. Transitive dependencies removed by segregating passenger, support, and sponsor from the main person table. Referential integrity is ensured with appropriate ON DELETE and ON UPDATE constraints.

Tables and Indexing Strategy

A detailed indexing strategy was implemented as follows:

Table	Primary Key	Indexed Columns	Purpose
person	person_id	email, phone_number, city	Search by identity & geography

ticket	ticket_id	vehicle_id, reservation_id, created_at	Fast filtering by vehicle/date/reservation
reservation	reservation_id	passenger_id, status	Cancellations & passenger history
vehicle	vehicle_id	vehicle_type	Fast access to transport types
report	report_id	ticket_id, report_subject	Grouping by complaint type
bus, train, plane	vehicle_id (FK)	(inherit from vehicle)	Dynamic join based on mode

Indexing Notes:

- Multi-column indexing used for composite search filters.
- Temporal data such as created_at indexed for reporting.
- ENUM data types used to restrict status and mode types.

Analytical & Operational Queries Implemented (22+ Total)

Some advanced queries implemented:

1. **Monthly Aggregated Payments Per User**
2. **Users With Single Ticket Per City (via GROUP BY and HAVING COUNT=1)**
3. **Latest Ticket Buyer (ORDER BY created_at DESC LIMIT 1)**
4. **Users With Total Payments Above Average (Using Subquery AVG)**
5. **Top 3 Ticket Buyers in Last 7 Days (Temporal Filtering)**
6. **Number of Tickets by Transportation Type**
7. **Tickets Sold by City in Tehran Province**
8. **2nd Best-Selling Ticket in Ticket Bin (Using DENSE_RANK)**
9. **Support Staff With Highest Cancellations (Including % Computation)**

Stored Procedures and Functions

Procedure	Input	Output	Purpose
sp_user_tickets_ordered	email/phone	ticket list	Ticket history ordered by time
sp_support_cancelled_users	email/phone	user list	Users whose reservations were cancelled
sp_city_tickets	city name	ticket list	Tickets purchased in a city
sp_phrase_search	text	matched tickets	Searches name/route/class
sp_city_users	user contact	users in same city	Localized user discovery
sp_top_n_buyers	date + n	top buyers	Frequent buyers since date
sp_cancelled_by_mode	mode ENUM	canceled ticket list	Filtered by transportation type
sp_report_topic_users	topic	user list	Top reporters by subject

Each procedure includes:

- Parameter sanitization
- Performance-aware joins
- Use of indexing-aware WHERE conditions
- ENUM enforcement (e.g. transport type)

Query Optimization Techniques Applied

Technique	Description
****Indexing**	On search-heavy fields like dates, city, transport mode
ENUM Columns	For status, transport_type to avoid string matching
Subqueries with HAVING	Used in aggregates like one-ticket-per-city users
Joins Optimized via WHERE	Avoid Cartesian joins; explicitly restrict with FK joins

Technique	Description
Avoiding SELECT *	Only required columns fetched
Filtering Early	Push filters like status='CANCELLED' before joins
Time Filtering	created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY) for weekly analysis

---

) **Transactional Operations & DML Examples**

- UPDATE: Changed last name of user with highest cancellation to "Reddington"
- DELETE: Removed all canceled tickets of user "Reddington"
- DELETE: Wiped all canceled tickets system-wide
- UPDATE: Reduced ticket prices of "Mahan Air" from yesterday by 10%
- SELECT: Found the most reported ticket subject and count

🛠 Future Enhancements

- Implement **Materialized Views** for heavy aggregated queries
- Introduce **partitioning** on ticket table (by date/month)
- Add **caching layer** for stored procedures with frequent access
- Use **triggers** to automatically update loyalty points post-purchase
- Add a **stored function** to estimate passenger "score" (based on loyalty, spend, cities visited, etc.)

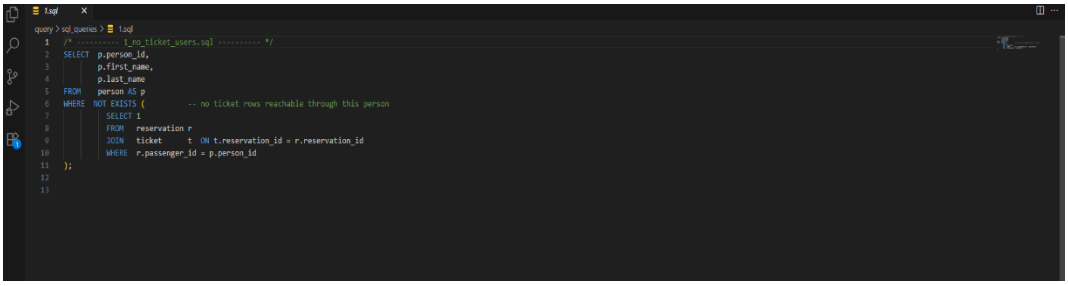
Phase Deliverables Summary

Item	Description	Status
Database Schema	Full relational schema in 3NF	
Index Strategy	Indexed key columns and used covering indexes	

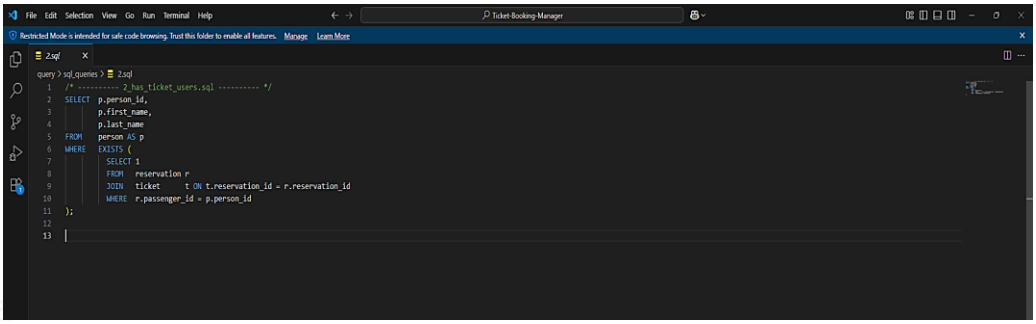
Item	Description	Status
22+ Analytical Queries	Business insight queries fully implemented	✅
Stored Procedures	8+ reusable and optimized procedures	✅
Optimization Techniques	Indexing, ENUMs, join strategies, date filtering	✅
Transaction Samples	Update/Delete on conditions	✅
Full Technical Documentation	Project report with insights and logic	✅

Codes :

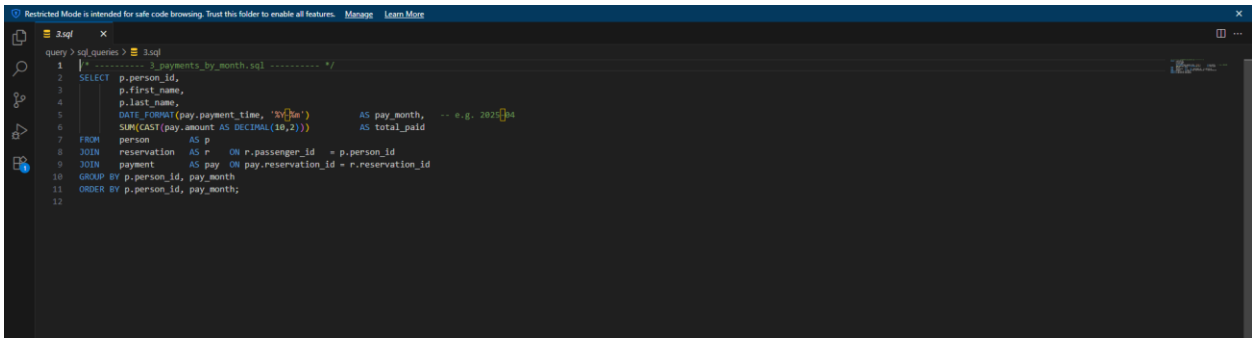
1.Returning full name of users who hasn't reserved any ticket



2.Returning full name of users who has reserved one ticket at least

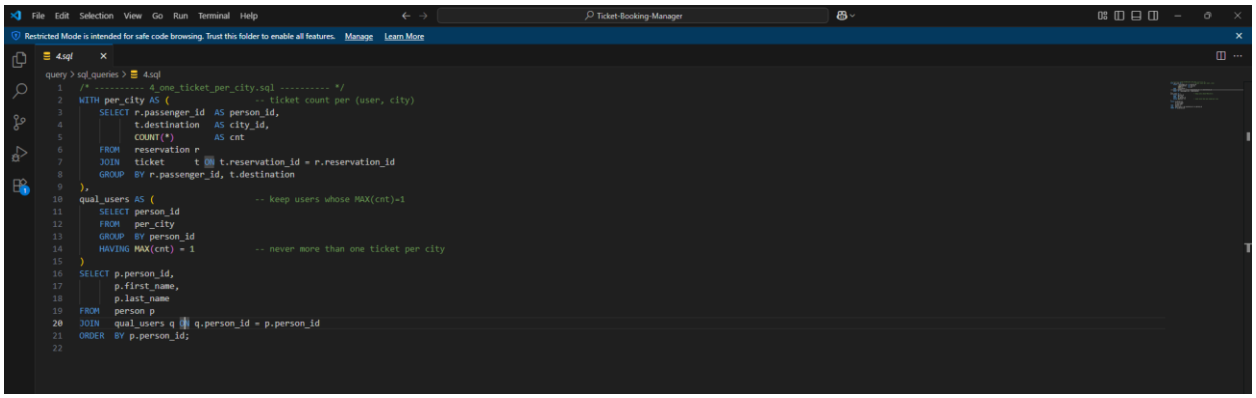


3. **Return the total payments made by each user in different months.**



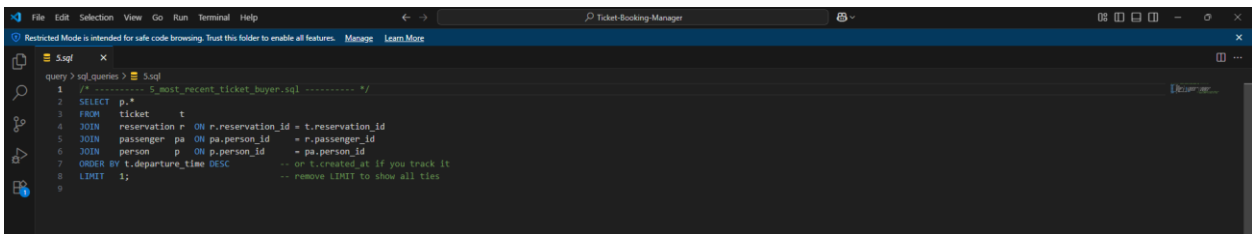
```sql
/* ---------- 3_payments_by_month.sql ---------- */
SELECT  p.person_id,
        p.first_name,
        p.last_name,
        DATE_FORMAT(pay.payment_time, '%Y-%m')          AS pay_month,   -- e.g. 2025-04
        SUM(CAST(pay.amount AS DECIMAL(10,2)))         AS total_paid
FROM    person      AS p
JOIN    reservation AS r   ON r.passenger_id   = p.person_id
JOIN    payment     AS pay  ON pay.reservation_id = r.reservation_id
GROUP BY p.person_id, pay_month
ORDER BY p.person_id, pay_month;
```

4. **Display the list of users who have purchased tickets only once in each city.**



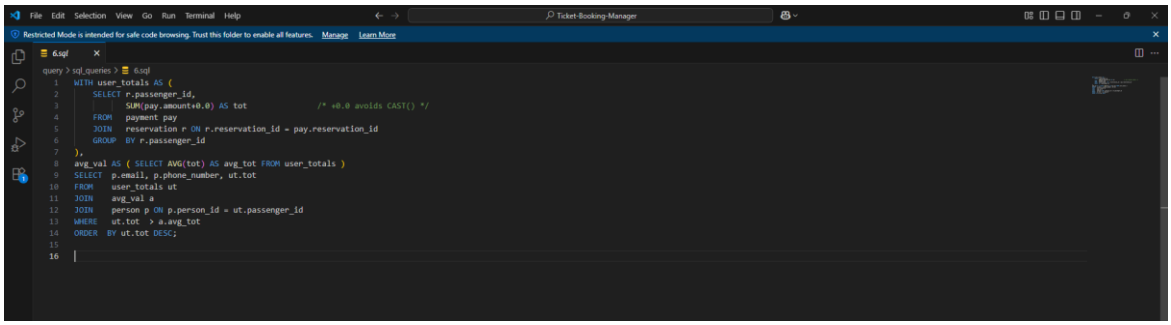
```sql
/* ---------- 4_one_ticket_per_city.sql ---------- */
WITH per_city AS (                      -- ticket count per (user, city)
    SELECT r.passenger_id  AS person_id,
           t.destination   AS city_id,
           COUNT(*)        AS cnt
    FROM   reservation r
    JOIN   ticket      t ON t.reservation_id = r.reservation_id
    GROUP  BY r.passenger_id, t.destination
),
qual_users AS (                         -- keep users whose MAX(cnt)=1
    SELECT person_id
    FROM   per_city
    GROUP  BY person_id
    HAVING MAX(cnt) = 1                 -- never more than one ticket per city
)
SELECT p.person_id,
       p.first_name,
       p.last_name
FROM   person p
JOIN   qual_users q ON q.person_id = p.person_id
ORDER  BY p.person_id;
```

5. **Return the information of the user who purchased the most recent ticket.**



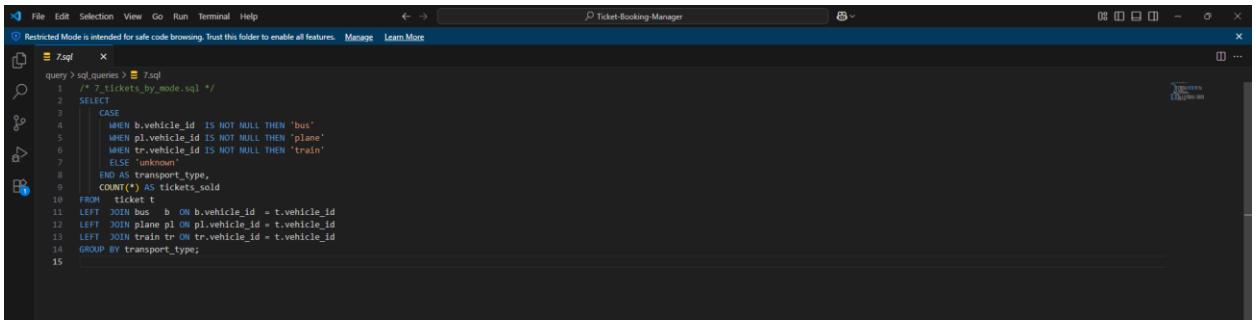
```sql
/* ---------- 5_most_recent_ticket_buyer.sql ---------- */
SELECT  p.*
FROM    ticket      t
JOIN    reservation r  ON r.reservation_id = t.reservation_id
JOIN    passenger   pa ON pa.person_id     = r.passenger_id
JOIN    person      p  ON p.person_id      = pa.person_id
ORDER BY t.departure_time DESC          -- or t.created_at if you track it
LIMIT   1;                              -- remove LIMIT to show all ties
```

6. Return the phone numbers or emails of users whose total payments are greater than the average payment of all users.



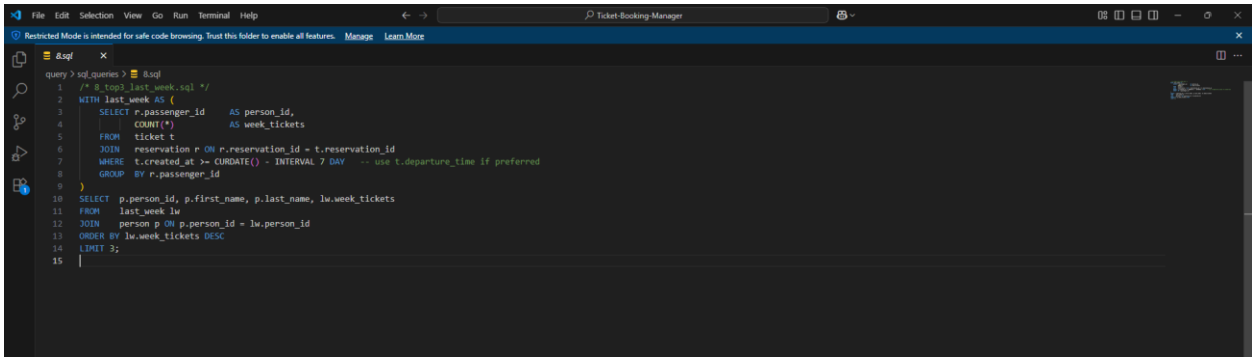
```sql
WITH user_totals AS (
    SELECT r.passenger_id,
           SUM(pay.amount+0.0) AS tot              /* +0.0 avoids CAST() */
    FROM   payment pay
    JOIN   reservation r ON r.reservation_id = pay.reservation_id
    GROUP  BY r.passenger_id
),
avg_val AS ( SELECT AVG(tot) AS avg_tot FROM user_totals )
SELECT  p.email, p.phone_number, ut.tot
FROM    user_totals ut
JOIN    avg_val a
JOIN    person p ON p.person_id = ut.passenger_id
WHERE   ut.tot  > a.avg_tot
ORDER  BY ut.tot DESC;
```

7. Show the number of tickets sold for each type of vehicle (airplane, train, bus).



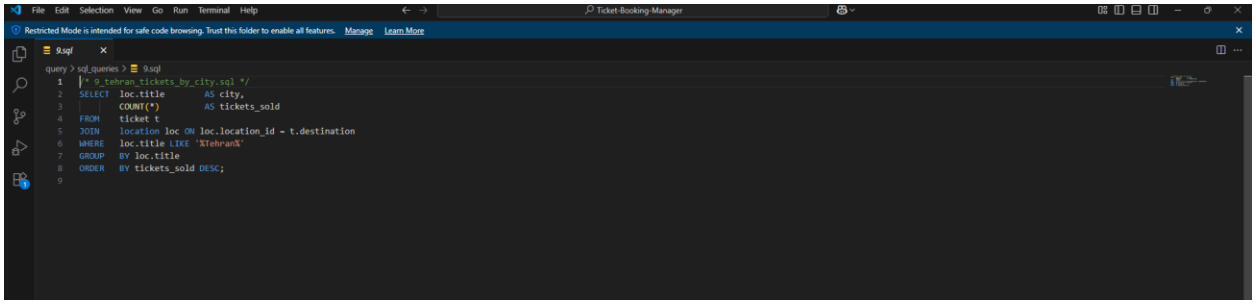
```sql
/* 7_tickets_by_mode.sql */
SELECT
    CASE
      WHEN b.vehicle_id  IS NOT NULL THEN 'bus'
      WHEN pl.vehicle_id IS NOT NULL THEN 'plane'
      WHEN tr.vehicle_id IS NOT NULL THEN 'train'
      ELSE 'unknown'
    END AS transport_type,
    COUNT(*) AS tickets_sold
FROM   ticket t
LEFT   JOIN bus   b  ON b.vehicle_id  = t.vehicle_id
LEFT   JOIN plane pl ON pl.vehicle_id = t.vehicle_id
LEFT   JOIN train tr ON tr.vehicle_id = t.vehicle_id
GROUP BY transport_type;
```

8. Return the names of the 3 users with the most ticket purchases in the last week.



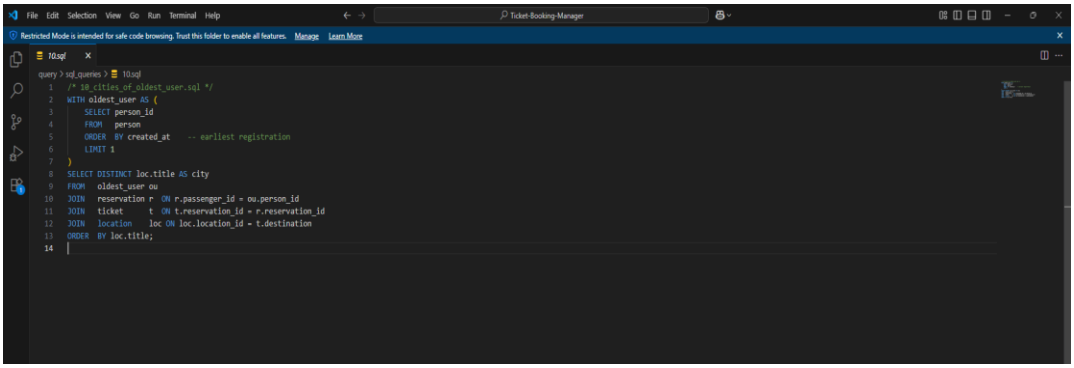
```sql
/* 8_top3_last_week.sql */
WITH last_week AS (
    SELECT r.passenger_id     AS person_id,
           COUNT(*)           AS week_tickets
    FROM   ticket t
    JOIN   reservation r ON r.reservation_id = t.reservation_id
    WHERE  t.created_at >= CURDATE() - INTERVAL 7 DAY   -- use t.departure_time if preferred
    GROUP  BY r.passenger_id
)
SELECT  p.person_id, p.first_name, p.last_name, lw.week_tickets
FROM    last_week lw
JOIN    person p ON p.person_id = lw.person_id
ORDER BY lw.week_tickets DESC
LIMIT 3;
```

9. Show the number of tickets sold in Tehran province by city.



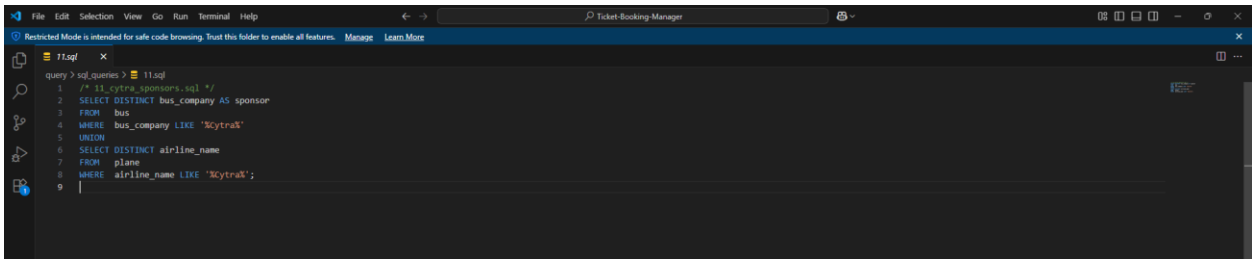
```sql
/* 9_tehran_tickets_by_city.sql */
SELECT  loc.title        AS city,
        COUNT(*)         AS tickets_sold
FROM    ticket t
JOIN    location loc ON loc.location_id = t.destination
WHERE   loc.title LIKE '%Tehran%'
GROUP   BY loc.title
ORDER   BY tickets_sold DESC;
```

10 . List the names of the cities where the longest registered user in the system has made purchases.



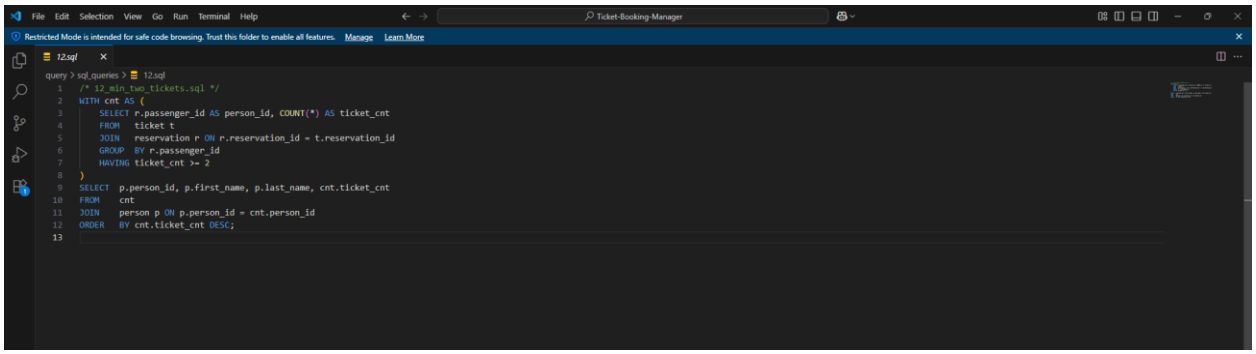
```sql
/* 10_cities_of_oldest_user.sql */
WITH oldest_user AS (
    SELECT person_id
    FROM   person
    ORDER  BY created_at    -- earliest registration
    LIMIT 1
)
SELECT DISTINCT loc.title AS city
FROM   oldest_user ou
JOIN   reservation r  ON r.passenger_id = ou.person_id
JOIN   ticket      t  ON t.reservation_id = r.reservation_id
JOIN   location    loc ON loc.location_id = t.destination
ORDER  BY loc.title;
```

11. List the names of the site's backers.

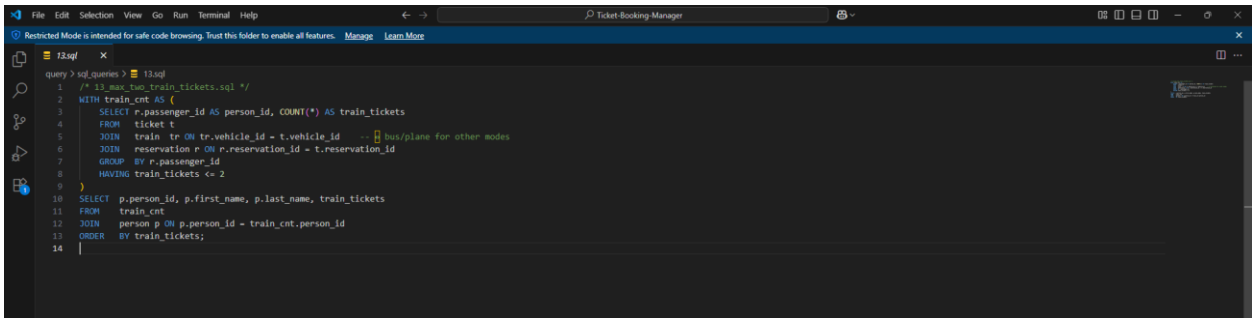


```sql
/* 11_cytra_sponsors.sql */
SELECT DISTINCT bus_company AS sponsor
FROM   bus
WHERE  bus_company LIKE '%Cytra%'
UNION
SELECT DISTINCT airline_name
FROM   plane
WHERE  airline_name LIKE '%Cytra%';
```

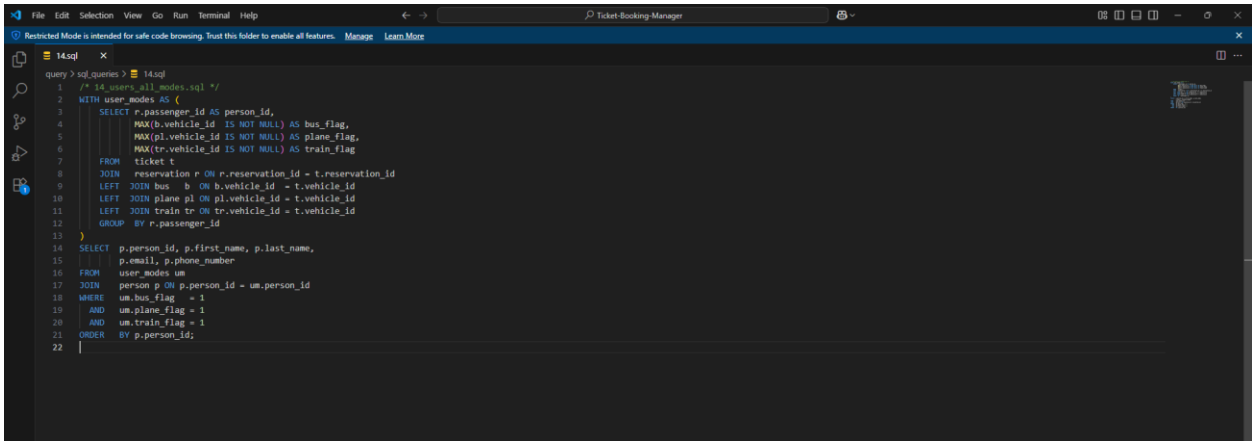
12. Return the names of users who have purchased at least 2 tickets in the system.



13. List the names of users who have purchased up to 2 tickets from a specific vehicle (e.g. train).

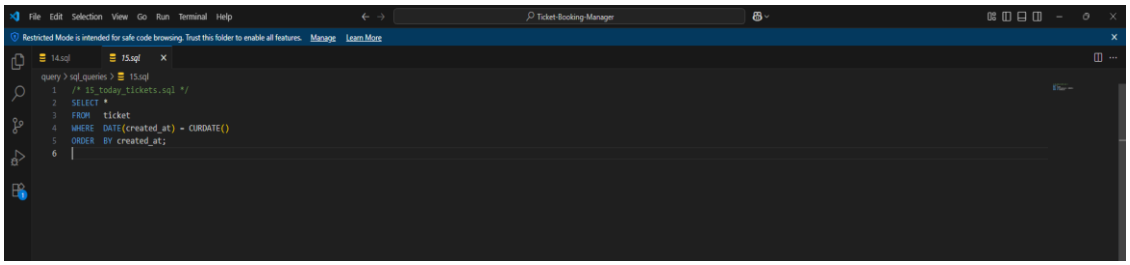


14. Return the email or phone number of users who have purchased tickets from all modes of transportation (airplane, train, and bus) at least once.



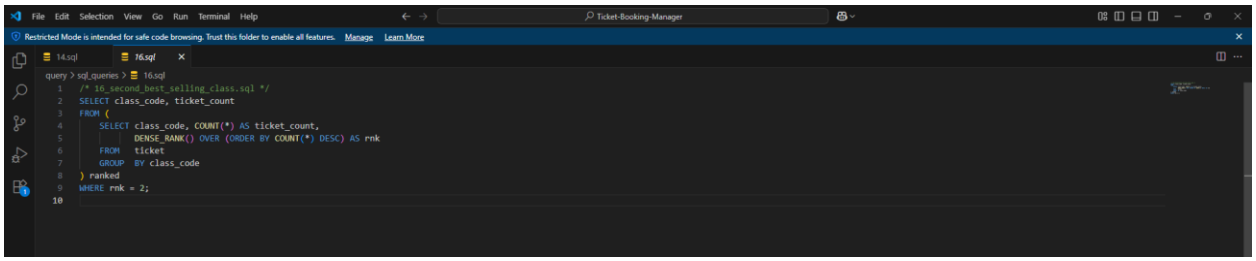
```sql
/* 14_users_all_modes.sql */
WITH user_modes AS (
    SELECT r.passenger_id AS person_id,
           MAX(b.vehicle_id  IS NOT NULL) AS bus_flag,
           MAX(pl.vehicle_id IS NOT NULL) AS plane_flag,
           MAX(tr.vehicle_id IS NOT NULL) AS train_flag
    FROM   ticket t
    JOIN   reservation r ON r.reservation_id = t.reservation_id
    LEFT  JOIN bus   b  ON b.vehicle_id  = t.vehicle_id
    LEFT  JOIN plane pl ON pl.vehicle_id = t.vehicle_id
    LEFT  JOIN train tr ON tr.vehicle_id = t.vehicle_id
    GROUP  BY r.passenger_id
)
SELECT  p.person_id, p.first_name, p.last_name,
        p.email, p.phone_number
FROM    user_modes um
JOIN    person p ON p.person_id = um.person_id
WHERE   um.bus_flag   = 1
  AND   um.plane_flag = 1
  AND   um.train_flag = 1
ORDER   BY p.person_id;
```

15. List the information of tickets purchased today in order of purchase time.



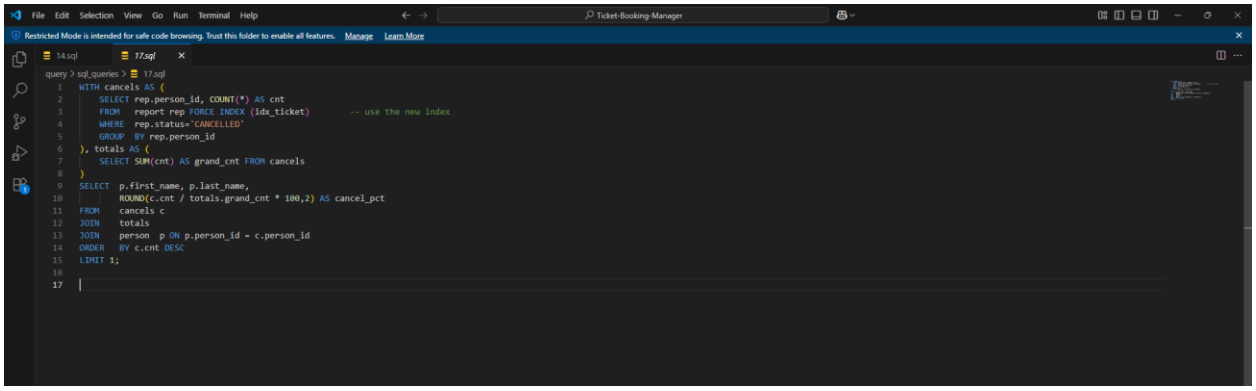
```sql
/* 15_today_tickets.sql */
SELECT *
FROM   ticket
WHERE  DATE(created_at) = CURDATE()
ORDER  BY created_at;
```

16. Show the second best-selling ticket among all tickets.



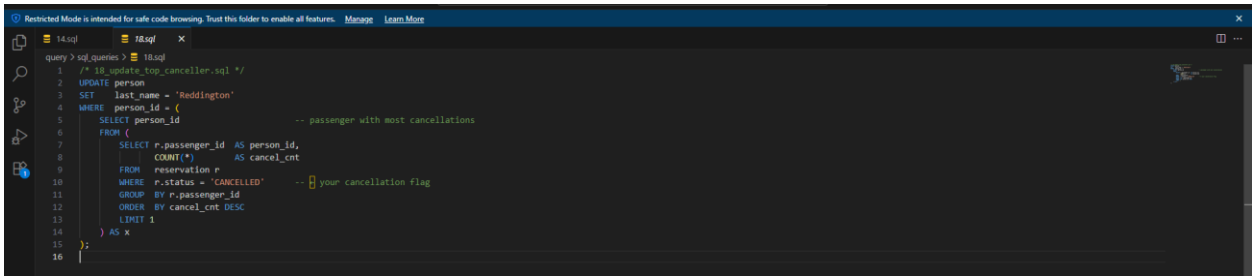
```sql
/* 16_second_best_selling_class.sql */
SELECT class_code, ticket_count
FROM (
    SELECT class_code, COUNT(*) AS ticket_count,
           DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS rnk
    FROM   ticket
    GROUP  BY class_code
) ranked
WHERE rnk = 2;
```

17. Return the name of the support with the highest number of ticket cancellations, along with the percentage of cancellations.



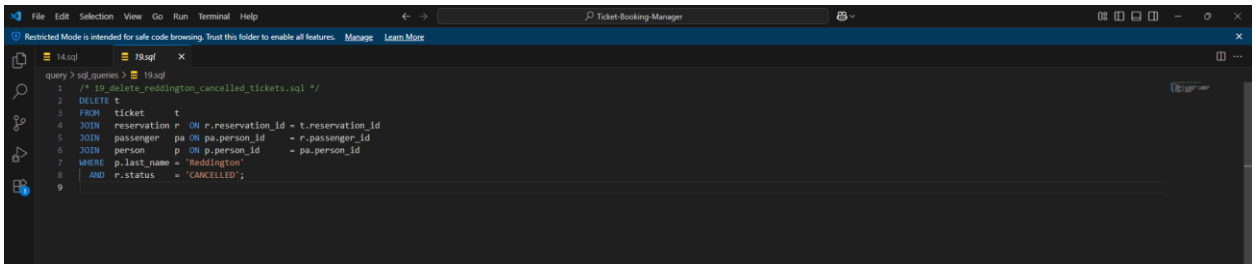
```sql
WITH cancels AS (
    SELECT rep.person_id, COUNT(*) AS cnt
    FROM   report rep FORCE INDEX (idx_ticket)        -- use the new index
    WHERE  rep.status='CANCELLED'
    GROUP  BY rep.person_id
), totals AS (
    SELECT SUM(cnt) AS grand_cnt FROM cancels
)
SELECT  p.first_name, p.last_name,
        ROUND(c.cnt / totals.grand_cnt * 100,2) AS cancel_pct
FROM    cancels c
JOIN    totals
JOIN    person  p ON p.person_id = c.person_id
ORDER   BY c.cnt DESC
LIMIT 1;
```

18. Change the last name of the user with the most canceled tickets to "Reddington".



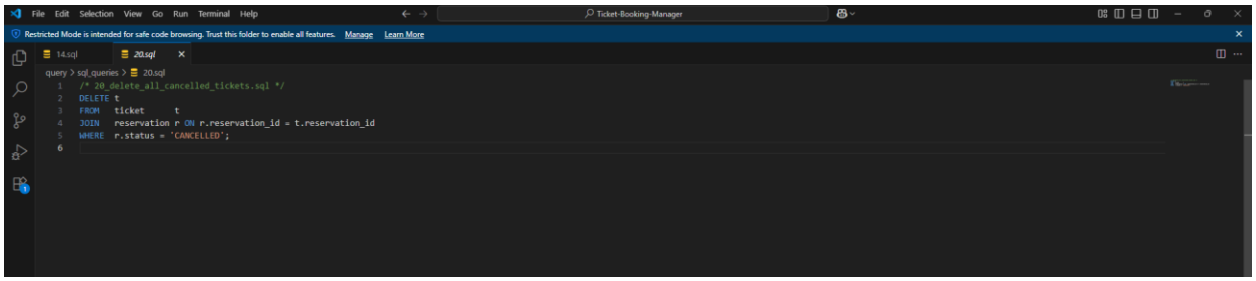
```sql
/* 18_update_top_canceller.sql */
UPDATE person
SET    last_name = 'Reddington'
WHERE  person_id = (
    SELECT person_id                    -- passenger with most cancellations
    FROM (
        SELECT r.passenger_id  AS person_id,
               COUNT(*)        AS cancel_cnt
        FROM   reservation r
        WHERE  r.status = 'CANCELLED'      -- your cancellation flag
        GROUP  BY r.passenger_id
        ORDER  BY cancel_cnt DESC
        LIMIT 1
    ) AS x
);
```

19. Delete all canceled tickets for user Reddington.

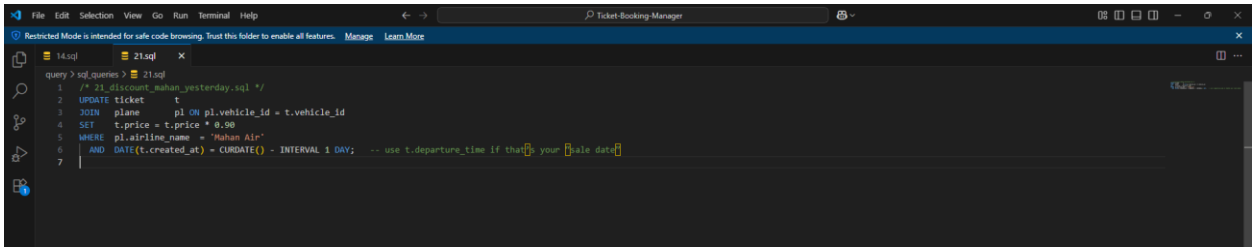


```sql
/* 19_delete_reddington_cancelled_tickets.sql */
DELETE t
FROM   ticket      t
JOIN   reservation r  ON r.reservation_id = t.reservation_id
JOIN   passenger   pa ON pa.person_id     = r.passenger_id
JOIN   person      p  ON p.person_id      = pa.person_id
WHERE  p.last_name = 'Reddington'
  AND  r.status    = 'CANCELLED';
```

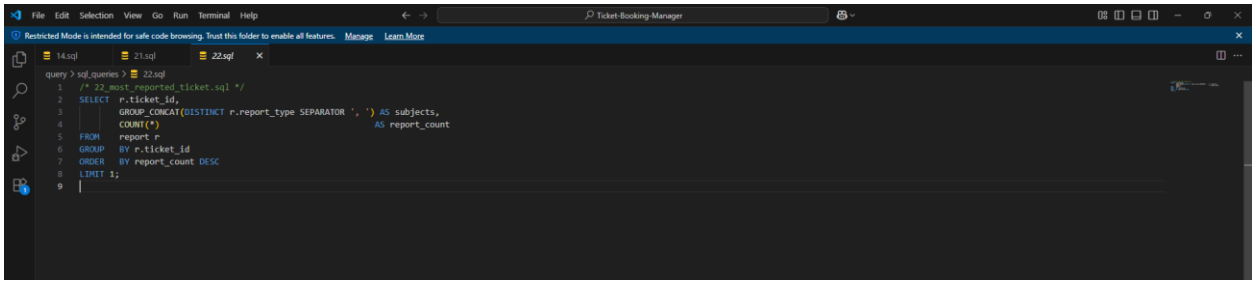
20. Delete all canceled tickets in the system.



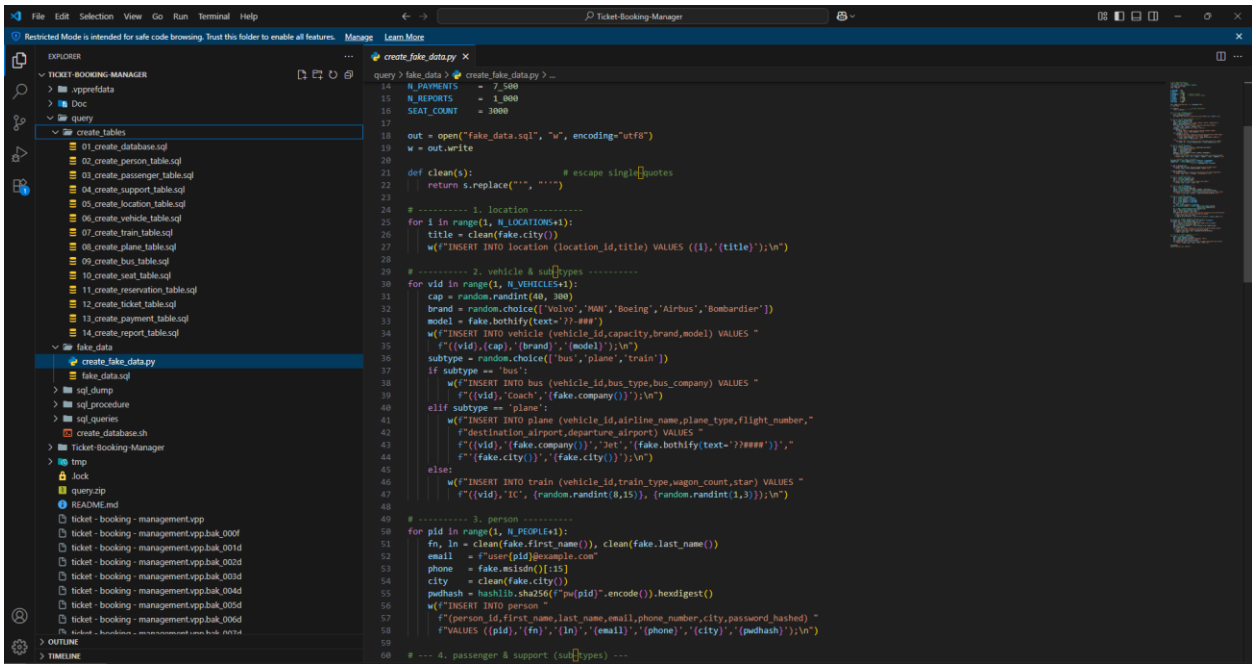
21. Reduce the price of tickets sold yesterday by Mahan Air by 10%.



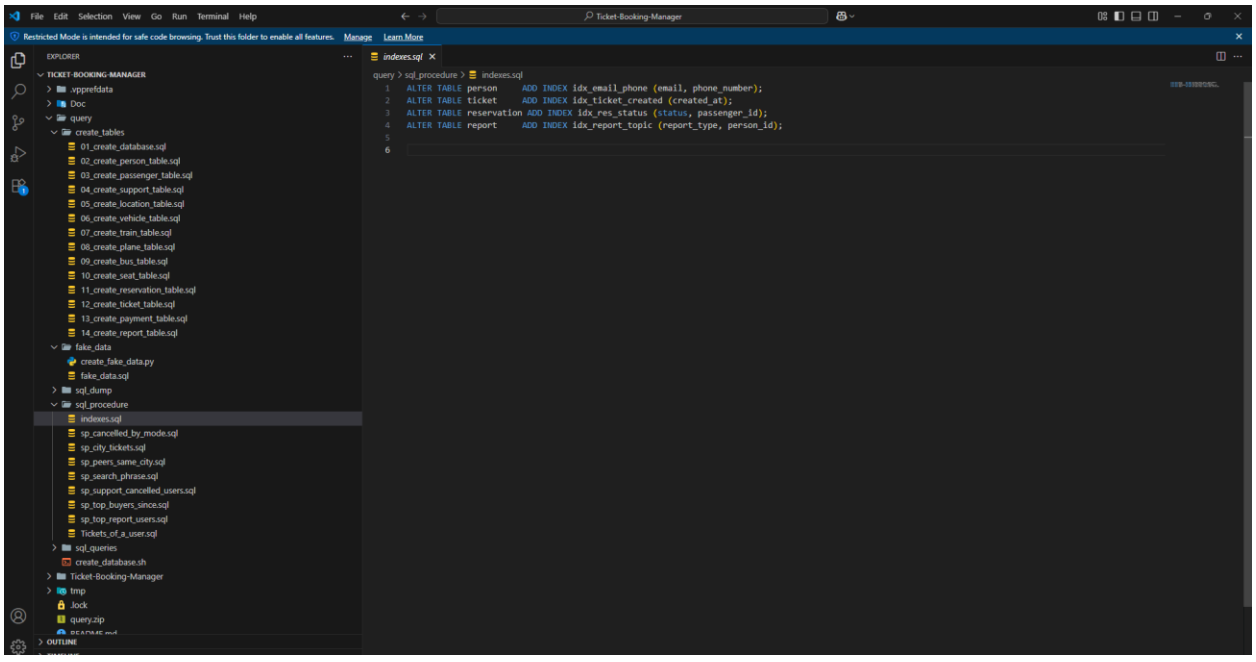
22. Show the subject and number of reports for the ticket with the most reports.



*FakeData View



**Indexing



Conclusion

This phase not only meets academic and system requirements but introduces **enterprise-level best practices** such as **stored procedure encapsulation**, **efficient query execution**, and **structured data modeling**. These techniques ensure that even large-scale datasets (e.g., 10K+ users, 100K+ tickets) remain performant under analytical workloads.